Sugar Labs

Google Summer of Code 2026

Project Information:

Name: Music Blocks Maintenance
Length: 150 hours
Coding Mentors: Walter Bender, Om Santosh Suneri

Student Details:

Full Name: Abhnish Kumar
Email: abhnish.1289@gmail.com
Github: abhnish
LinkedIn: abhnishkumar
Preferred Language: English
Location: New Delhi, India
Time Zone: Indian Standard Time (IST) (UTC +05:30)
Institution: Maharaja Agrasen Institute of Technology
Program: B.Tech in Computer Science and Technology
(Expected 2027)

About Me

I have always been fascinated by the idea of **building things** that create real-world impact. The thought of solving problems and improving lives using just a laptop felt like magic. This idea sparked my interest in programming and motivated me to explore how technology is built to create a meaningful impact.

I am currently pursuing my B.Tech in Computer Science & Technology at Maharaja Agrasen Institute of Technology, New Delhi, where I am ranked **11th** out of 180 students with a 9.0 GPA. Interestingly, a previous GSoC Sugar Labs contributor studied at the same institution, which further motivated me to apply.
My professional experience spans software development and testing internships. At Galvanic Infotech, I implemented and validated unit tests for assigned modules, improving overall test coverage by **20%**, and contributed to a **15%** reduction in repeated defects through disciplined QA practices. At SILVISH , I optimized JavaScript and CSS to improve website performance by **25%**. These experiences gave me a strong foundation in writing meaningful tests and delivering production-quality code.
Beyond internships, I have built full-stack projects including FinWise, an AI-powered financial reconciliation system using Node.js, PostgreSQL, and LLMs, and Public Service CRM, a civic tech CRM system built for the India Innovates 2026 hackathon by Municipal Corporation of Delhi, where I was selected in the **Top 500** out of **30,000+** participants competing in AI, Robotics, and Civic Tech tracks. These projects strengthened my skills in backend development, database design, and frontend engineering.
I co-authored and presented a research paper titled "Benchmarking the Trade-offs in Object Detection: Accuracy, Speed, and Energy Efficiency" at ICAIN-2025 (International Conference on AI and Networking), organized by BITS Pilani Dubai Campus in association with IIIT Allahabad and published with Springer.
In competitive programming, I have achieved a **Top 55** rank among **10,000+** participants in CodeClash on Unstop, hold a **3-star** rating on CodeChef , and am a **Specialist** on Codeforces reflecting my commitment to strong algorithmic thinking. Recently, I advanced to Round 1 of The **Big Code Challenge 2026 by Google**, qualifying among the Top students nationwide - reflecting my strong algorithmic problem-solving skills.
My open-source journey began with Sugar Labs' Music Blocks. What started as exploring the codebase quickly turned into deep involvement contributing tests, accessibility features, and code quality improvements across 20+ pull requests. The process of submitting pull requests, receiving feedback from maintainers like sum2it, omsuneri and engaging in threads alongside Walter Bender himself, has been an incredibly valuable and inspiring experience.

Sugar Labs- My Journey So Far

I discovered Sugar Labs through Music Blocks while exploring projects that combine software engineering with education and open-source learning. Music Blocks immediately stood out as a project where I could make real, measurable impact.

I started by understanding the codebase and submitted my first meaningful contribution removing the debug console statements from the jseditor widget (PR #5567). Since then, I have contributed consistently across the testing and accessibility, working on 20+ pull requests across two major issues.

Since then, I have been actively contributing to Music Blocks, working on bug fixes, feature improvements, and UI enhancements. I regularly attend community meetings and participate in the Sugar Labs and Music Blocks Matrix channel, discussing challenges, proposing ideas, and collaborating with maintainers.

My Contributions to Sugar Labs

Here is a detailed summary of the work and contributions that I did before the GSoC proposal period:

Features Added:

- **Removed debug console statements from jseditor widget (#5464)**
  - Set up the initial Cypress testing framework by creating the folder structure and writing the first set of test cases to automate UI testing. (Merged-#5567)
- **Fixed utils.js test coverage (#5945)**
  - Improved unit and branch coverage for js/utils/utils.js . (Closed-#6008)

Testing Contributions (Issue #5945 — Enhance Testing Suite):

- **Added branch coverage tests for musicutils.js -**
  - Improved coverage significantly with targeted branch tests. (PR-#6065 , #6275)
- **Added tests for NumberBlocks.js -**
  - Improved statement coverage from **55.96%** to **84.18%** and branch coverage from **46.42%** to **87.75%**. (PR-#6224)
- **Added tests for ActionBlocks.js -**
  - Improved statement coverage from **56.62%** to **92.05%** and line coverage from **58.53%** to **95.12%**. (PR-#6278)

- **Added tests for DrumBlocks.js -**
  - Added unit tests , improving branch coverage from **57.27%** to **80%+**. (PR-#6222)
- **Added tests for BooleanBlocks.js -**
  - Added tests for AND, OR, NOT, and comparison operator blocks, improving branch coverage from **35.34%** to **75%+**, covering null inputs and edge cases. (PR-#6217)
- **Added tests for DictBlocks.js -**
  - Added tests for dictionary operations including set, get, and error handling, improving coverage from **54.79%** to **80%+**. (PR-#6220)
- **Added tests for ExtrasBlocks.js -**
  - Added branch coverage tests for auxiliary utility blocks, improving coverage from **57.95%** to **80%+** including inStatusMatrix and error handling paths. (PR-#6218)
- **Added tests for OscilloscopeWidget -**
  - Added unit tests covering widget initialization, rendering, and cleanup methods under different audio contexts. (PR-#6214)
- **Added tests for ProgramBlocks.js -**
  - Added tests for start, action, and loop control blocks, improving coverage from **65.02%** to **80%+**. (PR-#6307)
- **Added tests for RhythmBlocks.js -**
  - Added tests for rhythm and beat block flow methods, covering musical timing logic and null connection handling. (PR-#6280)
- **Added tests for LogoDependencies.js -**
  - Added tests for dependency registration and lookup, improving coverage from **66.66%** to **80%+**. (PR-#6281)

Accessibility Contributions (Issue #5948 — WCAG 2.1 AA):

- **Added aria-labels to toolbar icons for screen reader support -**
  - Implemented aria-label and aria-pressed attributes across all toolbar icons. (PR-#6187 , # 6309)
- **Implemented high contrast mode toggle for WCAG 2.1 AA -**
  - Added invert_colors toggle with CSS theme, keyboard support, and localStorage persistence. (PR-#6189 , #6308)
- **Added skip link for canvas keyboard navigation -**
  - Allows keyboard users to skip directly to main content. (PR-#6196)
- **Added ARIA live regions for screen reader announcements -**
  - Dynamic announcements now reach screen reader users. (PR-#6195)
- **Added focus indicators for keyboard navigation -**

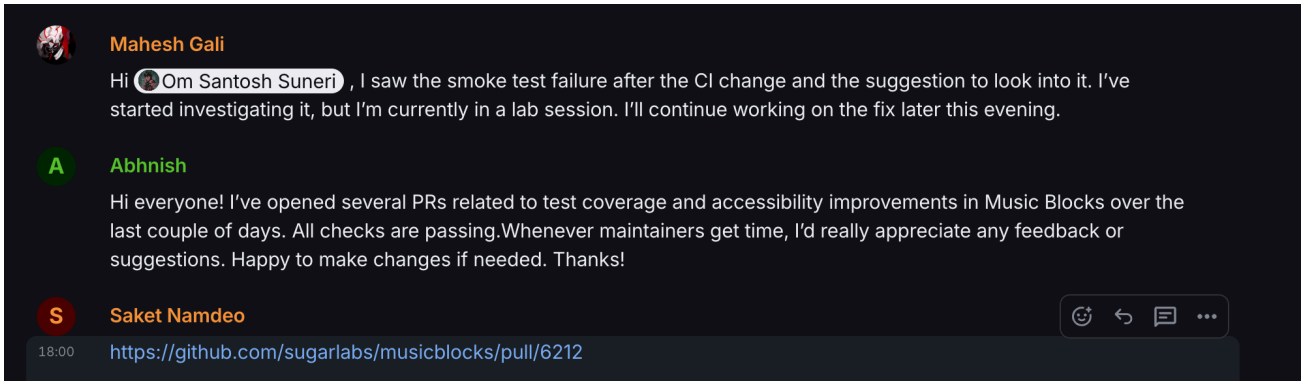
    - Implemented visible CSS focus rings across all interactive elements meeting WCAG 2.1 AA standards. (PR-#6193)
- **Added keyboard palette navigation -**
    - Enabled Tab/Enter keyboard navigation through the block palette without requiring a mouse. (PR-#6190)
- **Added widget close buttons accessibility-**
    - Added aria-labels and keyboard support to all widget close buttons for screen reader users. (PR-#6197)
- **Added aux toolbar accessibility -**
    - Added aria-labels and role attributes to all auxiliary toolbar icons for screen readers. (PR-#6198)
- **Added screen reader widget support -**
    - Added ARIA roles and keyboard interactions to key widgets for full screen reader support. (PR-#6194)
- **Demo Video:** (Accessibility Improvements - Link)

Community Engagement & Meetings:

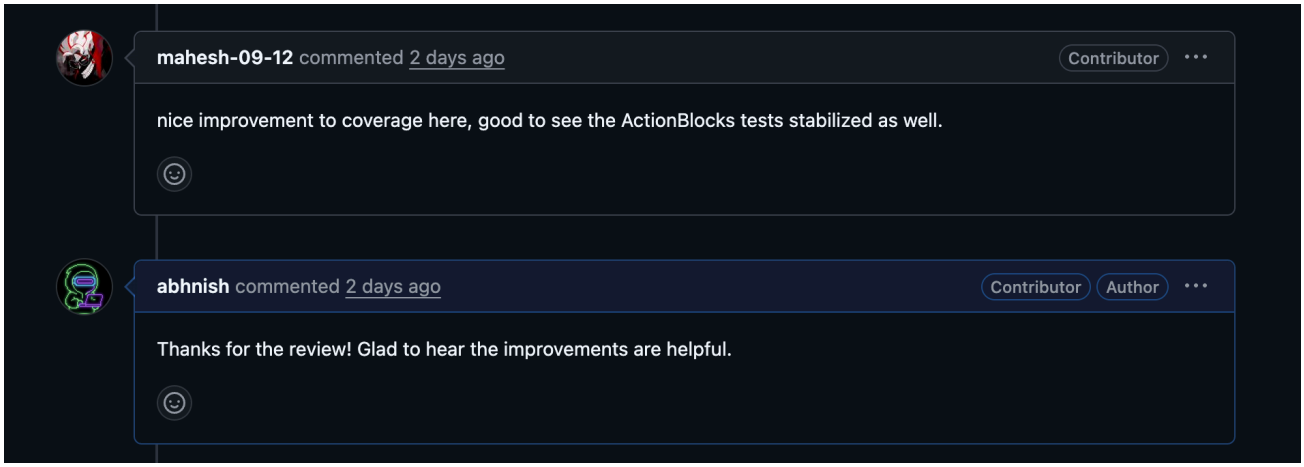
My Sugar Labs journey has been marked by active engagement with maintainers and contributors through GitHub discussions and the Matrix channel (@abhnish:matrix.org).
I have had direct interactions with key maintainers including: sum2it (Member) reviewed my accessibility PRs and requested a video demo of all accessibility changes. I recorded and delivered a comprehensive demo video covering :

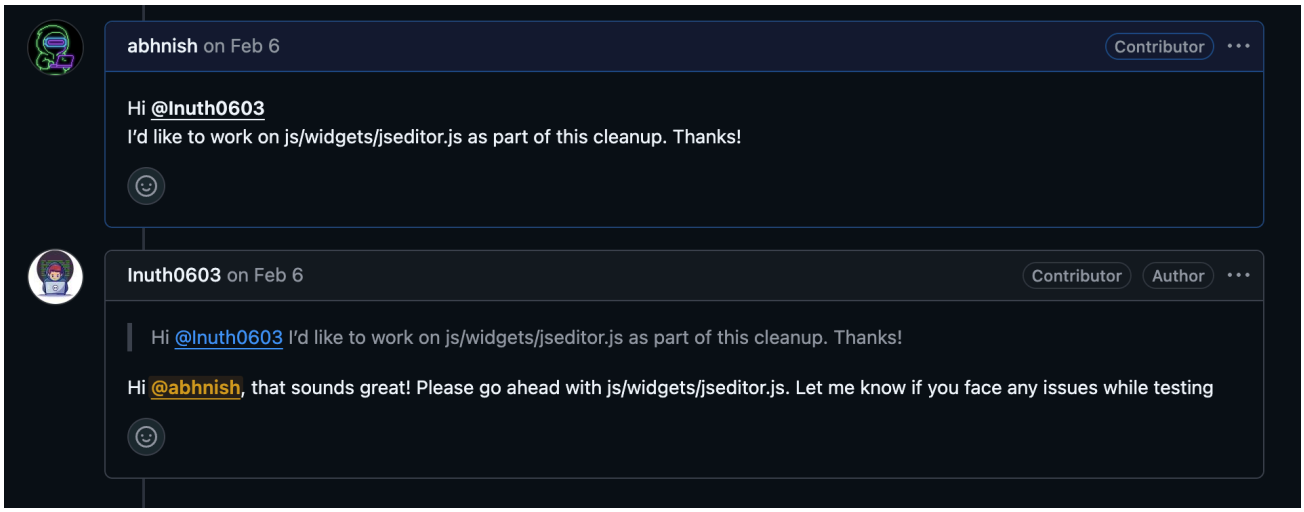
1. High contrast mode toggle .
2. Keyboard navigation through toolbar .
3. Palette keyboard navigation
4. ARIA labels verified in DevTools
5. Skip link navigation to main content
6. Help widget keyboard navigation All interactions were tested using keyboard navigation (Tab / Enter). Demo video link provided directly to sum2it. omsuneri (Member) provided feedback on my test PRs ,lnuth0603 (Contributor) - approved my request to work on js/widgets/jseditor.js as part of the code cleanup initiative, responding : "that sounds great! Please go ahead with js/widgets/jseditor.js. Let me know if you face any issues while testing". I investigated pre-existing CI failures, confirmed they were unrelated to my PRs by checking out upstream versions, and communicated this clearly with evidence . mahesh-09-12 (Contributor) reviewed my ActionBlocks test PR and noted: "nice improvement to coverage here, good to see the ActionBlocks tests stabilized as well."

Walter Bender (Founder) - active in issue #5607 (Music Blocks test suite project) where I contributed by summarizing the state of coverage and offering to help with block-level coverage verification, coverage tracker, and reviewing existing PRs.



I am also active in the Sugar Labs Matrix channel where I chat with contributors and maintainers, troubleshoot issues, and exchange ideas to improve the project.(My Matrix :- @abhnish:matrix.org)





omsuneri commented yesterday — Member

unrelated pr ?

omsuneri closed this yesterday

abhnish commented yesterday — Contributor · Author

Hi @omsuneri, I understand, the test fixes (palette.test.js, mathutils.test.js, turtle-singer.test.js) were added to fix pre-existing CI failures that were blocking the PR. I can reopen this with only the high contrast accessibility changes if that's preferred. Could you advise on the best approach for handling pre-existing test failures in CI?

manvirsingh01 mentioned this on Feb 9

Add coverage tracker for TurtleBlocks test suite #5636

abhnish on Feb 10 — Contributor

Thanks everyone — looks like all palettes from the original checklist are now covered

From what I can see, the core scope of #5607 is complete.
I'd be happy to help with next steps such as:

- verifying block-level coverage inside palettes,
- helping with the coverage tracker (Add coverage tracker for TurtleBlocks test suite #5636),
- improving test structure or documentation,
- or reviewing/testing existing PRs.

Please let me know where help would be most useful.

omsuneri on Feb 13 — Member

anyone working on this issue its suggested to please run the tests locally before you are pushing the changes !!

👍 1

Project Details

Introduction:

Music Blocks is an interactive web-based programming environment developed by Sugar Labs that helps students learn computational thinking through music. As the codebase grows in complexity, maintaining software quality becomes increasingly critical. Currently, Music Blocks faces three core maintenance challenges:

1. Insufficient test coverage many critical JavaScript modules have branch coverage below **60%**, making it difficult to catch regressions and validate new features.

2. Accessibility gaps Music Blocks is used by students of all abilities, but several UI components do not meet WCAG 2.1 AA standards, limiting access for users who rely on screen readers or keyboard navigation.

3. Code quality issues accumulated debug console statements, inconsistent formatting, and ESLint violations make the codebase harder to maintain and contribute to.

This project aims to systematically address all three challenges during GSoC 2026, building on the foundation I have already established through 20+ pull requests.

Current State of Testing in Music Blocks :

I am also active in the Sugar Labs Matrix channel where I chat with contributors and maintainers, troubleshoot issues, and exchange ideas to improve the project.

Through my pre-GSoC work, I have already improved coverage for NumberBlocks.js (**55% to 84%**), ActionBlocks.js (**56% to 92%**), musicutils.js, DrumBlocks.js, BooleanBlocks.js, DictBlocks.js, ExtrasBlocks.js, OscilloscopeWidget, ProgramBlocks.js, RhythmBlocks.js, and LogoDependencies.js.

During GSoC, I will extend this work to the remaining critical files.

Current State of Accessibility in Music Blocks Music Blocks is used by students of all ages and abilities. However, several WCAG 2.1 AA compliance gaps exist:

Current Limitations :

1. Many toolbar icons lack aria-labels, making them invisible to screen readers .

2. No high contrast mode for visually impaired users.

3. No keyboard-only navigation path through the canvas.

4. Widget close buttons not keyboard accessible.

5. No ARIA live regions for dynamic announcements.

6. Focus indicators missing on interactive elements.

Through my pre-GSoC accessibility contributions, I have already addressed aria-labels, high contrast mode, skip links, ARIA live regions, focus indicators, keyboard palette navigation, widget close buttons, aux toolbar accessibility, and screen reader widget support.

During GSoC, I will complete the remaining WCAG 2.1 AA compliance gaps and conduct thorough cross-browser screen reader testing.

Proposed GSoC Work

The GSoC project has three pillars :

Pillar 1: Complete the Test Suite

Building on Om Santosh Suneri's test suite work from 2025, I will complete coverage for the remaining critical files:

High Priority (complex, widely used):

1. turtle-singer.js (**37.74%**) - core music playback logic .

2. turtles.js (**14.25%**)- turtle management system .

3. toolbar.js (**48.94%**) - UI toolbar interactions .

4. GraphicsBlocks.js (**51.51%**) - graphics block operations.

Medium Priority:

1. turtle-painter.js (**43.41%**) .

2. turtledefs.js (**43.33%**).

3. themebox.js (**51.69%**).

4. remaining block files under **70%**.

For each file, I will follow the methodology I developed during pre-GSoC contributions:

1. Run coverage analysis to identify uncovered branches.

2. Study the source code to understand block logic .

3. Write tests validating real functional behavior .

4. Verify coverage improvement and ensure CI passes.

5. Document test patterns for future contributors .

Target: Achieve 75%+ branch coverage across all critical files.

Pillar 2: Package Dependency Updates

Music Blocks relies on several outdated packages that need review and updating :

1. Run npm outdated to audit all packages .

2. Prioritize packages with security vulnerabilities .

3. Update packages one at a time, running full test suite after each update .

4. Fix known breakages , e.g. latest jQuery breaks the search functionality .

5. Document all breaking changes and fixes in PRs .

Target: All critical packages updated to latest stable versions with no functionality regressions .

Technical Approach

Testing Strategy:

Music Blocks uses Jest as its testing framework. My approach, refined through 12+ test PRs, follows these principles:

1. Real behavior over branch coverage tests validate actual production logic, not just reach uncovered lines. As mahesh-09-12 (contributor) noted in my PR review: "nice improvement to coverage here, good to see the ActionBlocks tests stabilized as well ."

2. Minimal mocking I mock only external dependencies (activity, logo, turtles) and use real block instances via constructor interception to test actual flow() and arg() methods .

3. Edge case coverage null connections, inStatusMatrix branches, string input parsing, error throw paths, and backward singer branches are all covered .

4. CI compatibility all tests are verified locally before pushing, and pre-existing failures are identified and documented separately .

Accessibility Strategy:

1. Use axe-core for automated accessibility audits .

2. Manual keyboard-only navigation testing for all interactive elements .

3. Screen reader testing with VoiceOver on macOS .

4. ARIA implementation following WAI-ARIA 1.2 authoring practices .

5. Video demos provided to maintainers for visual verification (already done for pre-GSoC accessibility PRs) . (Demo Video - Link)

Coverage Improvement Plan (Visual) :

Already Improved (Pre-GSoC Contributions) :

File	Before	After
NumberBlocks.js	55.96%	84.18%
ActionBlocks.js	56.62%	92.05%
musicutils.js	~45%	78%
DrumBlocks.js	57.27%	80%
BooleanBlocks.js	35.34%	75%
DictBlocks.js	54.79%	80%
ExtrasBlocks.js	57.95%	80%
RhythmBlocks.js	low	improved
ProgramBlocks.js	65.02%	80%
LogoDependencies.js	66.66%	80%

Already Improved (Pre-GSoC Contributions) :

File	Current Coverage	Target Coverage
turtle-singer.js	37.74%	75%
turtles.js	14.25%	70%
toolbar.js	48.94%	80%
GraphicsBlocks.js	51.51%	80%
turtle-painter.js	43.41%	75%
turtledefs.js	43.33%	75%
themebox.js	51.69%	80%

Technical Requirements:

The implementation requires the following technologies:

- **Jest - Unit testing framework**
  - Used for all test coverage improvements across block files and utility modules .
- **JavaScript / HTML / CSS**
  - All accessibility improvements are implemented directly in the Music Blocks frontend codebase .
  - ARIA attributes, CSS focus rings, and keyboard event handlers added to existing components .
- **WAI-ARIA 1.2**
  - All ARIA implementations follow the official WAI-ARIA authoring practices specification .
- **ESLint + Prettier**
  - Used for code quality enforcement and consistent formatting across the codebase .
- **GitHub Actions**
  - CI pipeline integration for automated test coverage reporting on every PR .
- **axe-core accessibility auditing library**
  - Used for automated WCAG 2.1 AA compliance auditing across UI components .

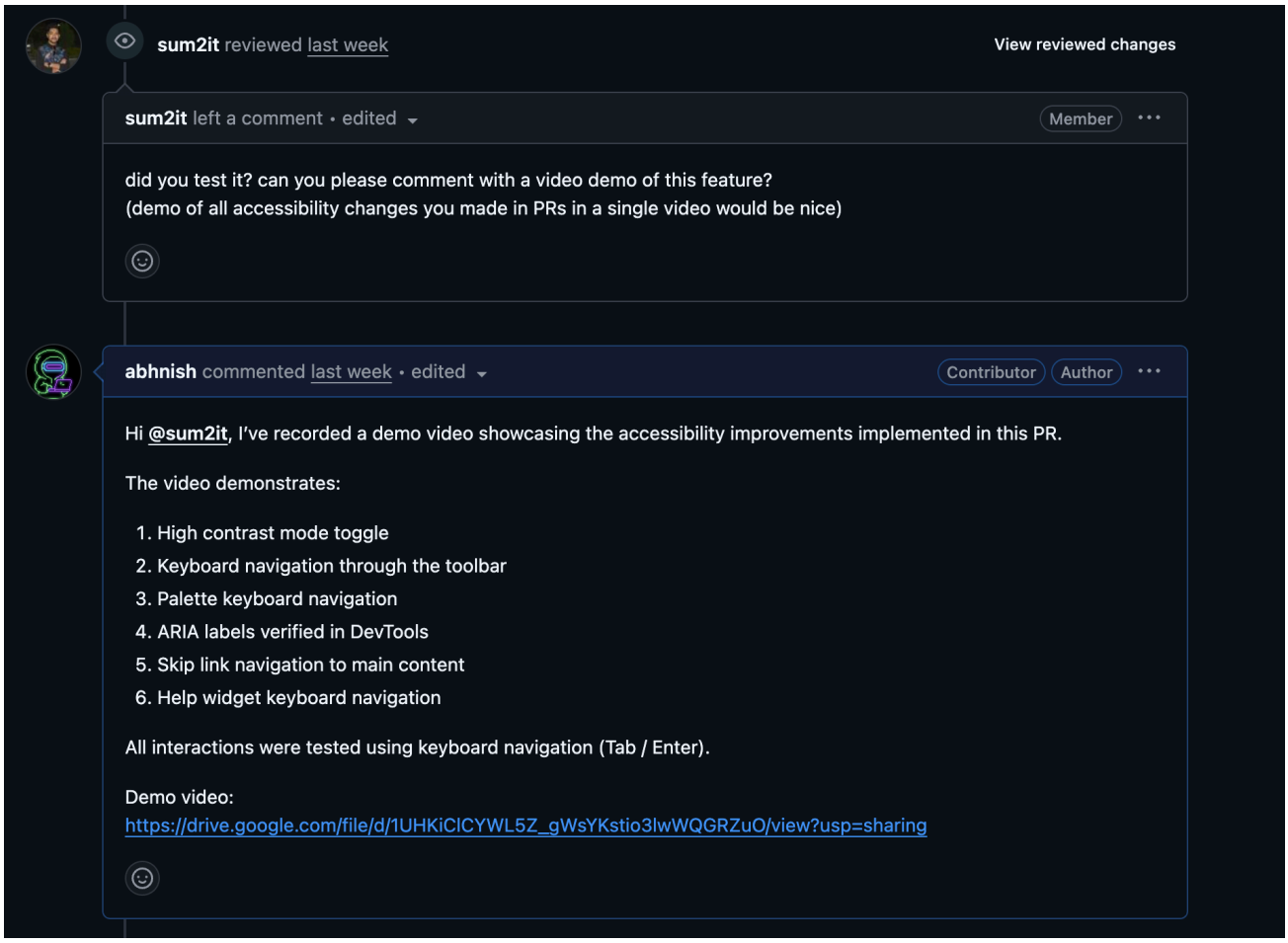
Frontend Integration:

The frontend improvements focus on two areas:

- **Accessibility Enhancements**
  - All toolbar icons, palette blocks, and widgets updated with proper ARIA attributes
    - Aria-label descriptive labels for all interactive elements .
    - Aria-pressed toggle state for buttons like high contrast mode .
    - Aria-live dynamic announcements for screen reader users .
    - Role attributes semantic roles for non-standard interactive elements .
  - Skip link added for keyboard users to bypass toolbar and reach main canvas directly .
  - Visible focus rings added across all interactive elements for keyboard navigation .

- **Developer Experience Improvements**
  - Contributor testing guide documenting :
    - How to identify low-coverage files .
    - Mock patterns for activity, logo, turtles .
    - Common pitfalls and how to avoid them .
    - How to run coverage locally before pushing .
  - WCAG 2.1 AA compliance checklist published as documentation for future contributors .
  - JSDoc documentation added to critical undocumented functions across key modules .

The implementation approach was developed iteratively through [20+ pull requests] and refined based on direct feedback from maintainers [sum2it], [omsuneri] .



omsuneri commented 2 days ago — Member

@abhnish resolve th etest failures !

abhnish commented 2 days ago — Contributor · Author

Hi @omsuneri ! These test failures appear to be pre-existing failures in `palette.test.js`, `ActionBlocks.test.js`, `turtle-singer.test.js`, and `mathutils.test.js` that are unrelated to this PR - this PR only adds `LogoDependencies.test.js`. I'll investigate and fix any failures caused by my changes. Could you confirm if these tests were already failing on `master` before my PR?

mahesh-09-12 commented 2 days ago — Contributor

@abhnish thanks for working on improving coverage here. Since the maintainer mentioned the focus is more on functional coverage rather than branch coverage, it might be worth ensuring these tests are validating real behavior instead of just covering branches. Also, this PR introduces additional failing tests in CI on top of existing ones, which makes it harder to isolate issues, it would be good to ensure new tests pass independently and to run the full test suite locally before pushing.

abhnish commented 2 days ago · edited — Contributor · Author

Hi @mahesh-09-12 , thank you for the feedback!
Regarding functional coverage , the tests do validate real behavior: they verify that `MakeBlockBlock.arg()` correctly constructs block stacks for named block types (`note` , `start` , `silence` , `tempo` , `volume` , `instrument`), handles missing blocks with errors, and validates argument type mismatches. Each test exercises the actual production logic, not just branch paths.
Regarding CI failures - the failing tests (`palette.test.js` , `ActionBlocks.test.js` , `turtle-singer.test.js` , `mathutils.test.js`) are pre-existing failures on `upstream/master` unrelated to this PR. I've verified this by checking out upstream versions of these files and confirming the failures exist independently. I will try to fix `ActionBlocks.test.js` failures in this PR.

Deliverables

By end of GSoC 2026, I will deliver :

- **Testing :**
  - **75%+ branch coverage across all the critical Music Blocks modules .**
  - **Comprehensive test suite covering turtle-singer.js, turtles.js, toolbar.js, GraphicsBlocks.js, and remaining block files .**
  - **Contributor testing guide documenting patterns and best practices .**
  - **CI coverage reporting integration** .
- **Accessibility :**
  - **A focused set of high-priority WCAG 2.1 AA improvements for the Music Blocks UI .**
  - **A Color contrast audit with fixes for the most important violations .**
  - **Verification of accessibility behaviour in major browsers and with at least the most relevant assistive technology combinations for Music Blocks .**
  - **Documentation of accessibility improvements and guidance for future contributors** .
- **Code Quality :**
  - **ESLint violations resolved across flagged files .**
  - **JSDoc documentation added to critical undocumented functions .**
  - **Debug console statements cleaned up .**
  - **Code quality report documenting improvements .**

- **Challenges and Mitigations :**
  - **Challenge 1:** Complex files like turtle-singer.js and turtles.js have deep dependencies and global state .
  - **Mitigation:** I have already developed robust mocking patterns for activity, logo, and turtles objects through my pre-GSoC work. I will extend these patterns progressively .

  - **Challenge 2:** Pre-existing test failures in CI make it hard to isolate new failures .
  - **Mitigation:** I have experience identifying and documenting pre-

existing failures (as demonstrated in my PR #6281 discussions with omsuneri). I will always run the full test suite locally before pushing.Deploy the system & integrate it into Sugar Labs' infrastructure .

- **Challenge 3:** Accessibility verification requires careful testing across browsers and assistive technologies .
- **Mitigation:** I will focus on the highest impact accessibility issues first and validate them in the browsers and screen reader setups that are most relevant to Music Blocks. I have already demonstrated accessibility-related changes through working demos during my pre-GSoC contributions .

- **Challenge 4:** Scope creep could make the project too broad .
- **Mitigation:** I will keep testing as the primary goal, treat accessibility as a targeted secondary goal, and limit code-quality work to files directly affected by the project .

My pre-GSoC work serves as a working proof of concept for the full GSoC project demonstrating my testing methodology, accessibility implementation approach, and ability to collaborate effectively with the Sugar Labs community .

Timeline

This timeline provides a structured roadmap for completing the Music Blocks test suite and updating package dependencies. While the plan may evolve based on mentor feedback, it ensures clear progression toward both project goals.

- **Pre-GSoC Period (Now - May 2026)**
  - Continue improving test coverage for remaining low-coverage files .
  - Study Om Santosh Suneri's existing test suite work to understand what has been completed and what remains .
  - Run npm outdated to get initial package audit .
  - Resolve any pending PR feedback from maintainers .
  - Study turtle-singer.js and turtles.js architecture in preparation for GSoC testing work .

- **Community Bonding Period (May 8 - June 1, 2026)**
  - Engage with mentors Walter Bender and Om Santosh Suneri to finalize work priorities .
  - Complete full package dependency audit using npm outdated and npm audit .

- Identify and document all pre-existing test failures in CI to establish a clean baseline .
- Map remaining uncovered files against Om Santosh's completed work to avoid duplication .
- Review recently merged PRs that may affect planned work .

- **Phase 1: Complete Test Suite (June 2 - July 14, 2026)**
  - **Week 1-2 (June 2 - June 15)**
    - Write comprehensive tests for toolbar.js (**48.94%** to **80%+**) .
    - Write tests for GraphicsBlocks.js (**51.51%** to **80%+**) .
    - Run full coverage analysis after each PR to track progress .
    - Ensure all new tests pass CI independently.
  - **Week 3-4 (June 16 - June 30)**
    - Write comprehensive tests for turtledefs.js (**43.33%** to **75%+**) .
    - Write tests for themebox.js (**51.69%** to **80%+**) .
    - Begin analyzing turtle-painter.js for test strategy .
    - Midpoint documentation update .

  - **Week 5-6 (July 1 - July 14)**
    - Write tests for turtle-painter.js (**43.41%** to **75%+**) .
    - Begin turtle-singer.js test strategy and initial tests .
    - This is the most complex file significant research required .

- **Midterm Deliverables (July 14 - July 18, 2026):**
  - **Deliverables:**
    - toolbar.js, GraphicsBlocks.js, turtledefs.js, themebox.js all at **75%+** coverage .
    - turtle-painter.js and turtle-singer.js substantially improved .
    - All new tests passing in CI .
    - Progress documentation updated .
    - Package audit report ready for Phase 2 .

- **Phase 2: Package Dependency Updates (July 15 - August 18, 2026)**
  - **Week 7-8 (July 15 - July 28)**
    - Complete turtle-singer.js tests (**37.74%** to **75%+**) .
    - Complete turtles.js tests (**14.25%** to **70%+**) .
    - Start package updates , begin with packages flagged by npm audit for security vulnerabilities .
    - Update packages incrementally, running full test suite after each update .

- **Week 9-10 (July 29 - August 11)**
    - Complete turtles.js tests .
    - Fix jQuery breakage causing search functionality to fail with latest jQuery version .
    - Continue updating remaining outdated packages .
    - Document all breaking changes and their fixes .
- **Week 11-12 (August 12 - August 18)**
    - Verify all package updates pass full test suite .
    - Fix any remaining functionality regressions caused by package updates .
    - Resolve pre-existing ESLint violations in flagged files .

- **Phase 3: Documentation & Polish (August 19 - September 1, 2026)**
    - **Week 13-14 (August 19 - September 1)**
        - Write contributor testing guide documenting :
            - How to identify low-coverage files .
            - Mock patterns for activity, logo, turtles .
            - Common pitfalls and how to avoid them .
            - How to run coverage locally before pushing
        - Document package update process for future maintainers .
        - Final coverage report showing before/after improvements .
        - Set up coverage reporting in CI pipeline .

- **Final Evaluation Deliverables (August 25 - September 1):**
    - **75%+** branch coverage across all targeted files .
    - Test suite complete building on Om Santosh's 2025 work .
    - All critical packages updated to latest stable versions .
    - jQuery search functionality fixed .
    - Contributor testing guide published .
    - Package update documentation published .
    - CI coverage reporting integrated .
    - Final blog post summarizing all GSoC work .

Availability

I am fully committed to Sugar Labs and this project for the duration of GSoC 2026. I am flexible with working hours and can adapt my schedule to match mentor availability across time zones .

- Weekly commitment : 15 - 20 hours per week

- Working hours: Flexible, available across morning to late evening IST .
- Most productive days: Monday through Saturday .
- Time Zone: Indian Standard Time (IST) (UTC +05:30) .

If I have some personal commitments during the May-August period, but I will communicate any schedule changes to mentors well in advance and ensure steady progress is maintained at all times. I am committed to delivering all milestones on time regardless of personal commitments .

Progress Reporting

- Daily updates on the Sugar Labs Matrix channel sharing key developments and blockers .
- Weekly summary comments on my open PRs .
- Participation in bi-weekly community meetings with mentors .
- Blog post every 2-3 weeks documenting technical progress, key learnings, and next steps .

Post-GSoC Plans

After completing GSoC, I plan to continue as a long-term contributor to Sugar Labs and Music Blocks. My specific post-GSoC goals are :

- Maintain the test suite I built during GSoC, reviewing new PRs that may reduce coverage and helping contributors write tests for new features .
- Help new contributors get started with Music Blocks testing the contributor guide I write during GSoC will serve as the foundation for this .
- Extend accessibility work to Music Blocks as development progresses, ensuring the new version maintains WCAG 2.1 AA compliance from the start .
- Explore contributing to other Sugar Labs projects, particularly around test infrastructure and accessibility .
- Continue engaging with the Sugar Labs community on Matrix and in bi-weekly meetings .

I see this GSoC project not as a one-time contribution, but as the beginning of a long-term relationship with Sugar Labs. The mission of making education open, transparent, and accessible resonates deeply with me, and I want to help sustain and grow this community.

Conclusion

Thank you for taking the time to read my proposal .

Music Blocks is a powerful educational tool used by students worldwide, and its long-term success depends on a robust, well-tested, and accessible codebase. This project directly addresses the most pressing maintenance needs test coverage, WCAG 2.1 AA accessibility, and code quality with a clear plan, proven methodology, and strong evidence of pre-existing contribution.

I am not starting from zero. I have already submitted 20+ pull requests, received positive feedback from maintainers including sum2it and omsuneri , engaged in threads alongside Walter Bender, and delivered video demos of my accessibility work. I understand the codebase, the testing patterns, and the community expectations.

With my background in software testing (**20%** coverage improvement at Galvanic Infotech internship), full-stack development, and competitive programming, combined with my deep familiarity with the Music Blocks codebase, I am confident in delivering a meaningful, lasting improvement to Sugar Labs. My focus is entirely on Sugar Labs, and I am fully committed to contributing to this project without any plans to submit proposals elsewhere.